

Локальный Lakehouse — пошаговый гайд для НОВИЧКОВ

Стек: **MinIO** → **Iceberg** (через **Lakekeeper REST Catalog**) → **Trino**.

Это гайд для тех, кто ни разу не трогал ни Docker, ни Lakehouse. Каждый этап — одна команда `docker run` (без `docker-compose` и без скриптов). Всё проверено на Docker Desktop (macOS/Windows).

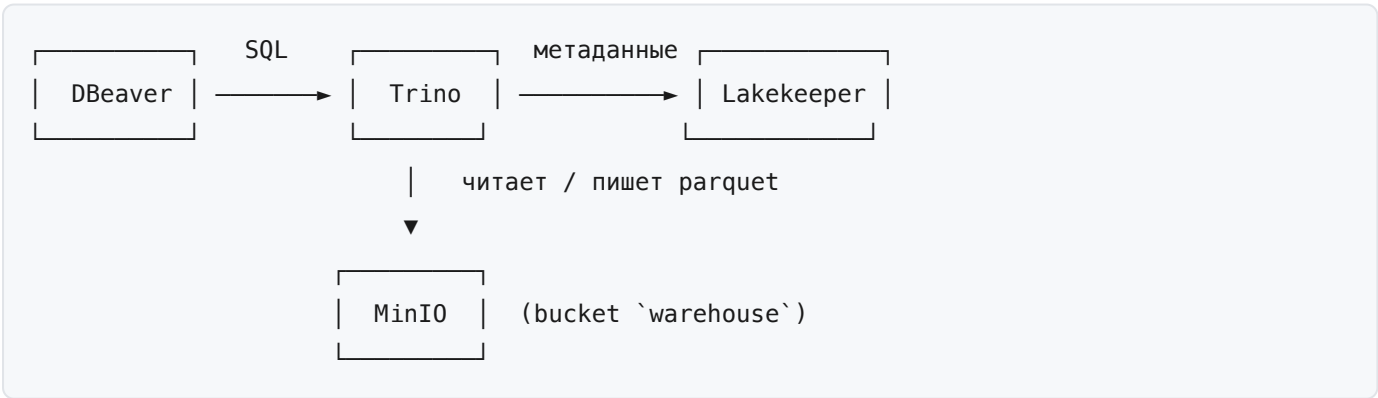
0. Что мы строим и зачем

Lakehouse — это когда данные лежат в *объектном хранилище* (как в data lake), но работать с ними можно через обычный **SQL** и таблицы (как в data warehouse).

Три компонента, каждый — отдельный Docker-контейнер:

Компонент	Что делает	Аналог в облаке
MinIO	S3-совместимое объектное хранилище. Сюда физически падают файлы данных (parquet).	AWS S3
Lakekeeper	Iceberg REST Catalog. Хранит метаданные таблиц (какие файлы/снапшоты относятся к таблице).	AWS Glue / Nessie
Trino	SQL-движок. Ты пишешь SQL, Trino ходит в Lakekeeper за метаданными и читает/пишет parquet в MinIO.	Athena / Presto

Схема:



1. Пререквизиты (проверь один раз)

1.1. Docker Desktop. Скачай и установи: <https://www.docker.com/products/docker-desktop/>. Запусти приложение и дождись, пока значок кита перестанет «думать».

Проверь в терминале:

```
docker --version          # должно вывести что-то вроде Docker version 27.x
docker run --rm hello-world # должно напечатать "Hello from Docker!"
```

Если `hello-world` отработал — Docker готов.

1.2. Свободные порты. Мы займём порты `9000`, `9001`, `5432`, `8181`, `8080`. Убедись, что на них никто не сидит:

```
lsof -nP -iTCP:9000 -iTCP:9001 -iTCP:5432 -iTCP:8181 -iTCP:8080 -sTCP:LISTEN
```

Пустой вывод = порты свободны.

⚠ **Почему фильтр `-sTCP:LISTEN`?** Без него команда может показать кучу строк вида `com.docke ... localhost:57157->localhost:8181 (CLOSE_WAIT)` — это не занятые порты, а «хвосты» сокетов от Docker после остановки контейнеров (соединения в состоянии закрытия). Нас интересуют только строки с `(LISTEN)` — то есть реально работающие сервисы. Флаг `-sTCP:LISTEN` показывает только их. Флаг `-nP` выводит адреса и порты цифрами (без служебных имён вроде `cslistener`).

1.3. Что такое `docker run`. Одна команда скачивает образ (если его нет локально) и запускает контейнер. Флаги, которые будут встречаться:

- `-d` — запустить в фоне (detached)
- `--name имя` — дать контейнеру понятное имя
- `-p ХОСТ:КОНТЕЙНЕР` — пробросить порт с контейнера на твой компьютер
- `-e КЛЮЧ=ЗНАЧЕНИЕ` — переменная окружения (пароли, настройки)
- `-v имя:/путь` — том для хранения данных (переживает перезапуск контейнера)
- `--rm` — удалить контейнер после завершения (для одноразовых команд)

1.4. Рабочая папка. Все файлы лабораторной (конфиги) держим в одной папке. Создай её в домашней директории и перейди в неё:

```
mkdir -p ~/lakehouse
cd ~/lakehouse
```

⚠ **Почему не `mkdir /lakehouse`?** Путь `/` — это корень файловой системы. На macOS (и Linux) он только для чтения — отсюда ошибка `Read-only file system`, и `sudo` не поможет. Папки создаём у себя в `~` — это твоя домашняя директория (она же `/Users/твой_логин`). Дальше все команды с `$(pwd)` и `@create-warehouse.json` выполняются **из этой папки** (`~/lakehouse`). Сюда же мы положим файл `create-warehouse.json` и папку `catalog/` (создадим их по ходу гайда).

2. Шпаргалка: порты и пароли

Сервис	Адрес	Логин	Пароль
MinIO — консоль (UI)	http://localhost:9001	minioadmin	minioadmin
MinIO — S3 API	http://localhost:9000	minioadmin	minioadmin
Postgres	localhost:5432	postgres	postgres
Lakekeeper — REST API	http://localhost:8181	—	без авторизации
Trino	http://localhost:8080	любой (например trino)	пусто

Все пароли — демо, только для локальной лабораторной.

3. Этап 1 — MinIO (объектное хранилище)

Запускаем MinIO одной командой:

```
docker run -d --name lakehouse-minio \  
-p 9000:9000 -p 9001:9001 \  
-e MINIO_ROOT_USER=minioadmin \  
-e MINIO_ROOT_PASSWORD=minioadmin \  
-v minio-data:/data \  
quay.io/minio/minio:latest server /data --console-address ":9001"
```

Обрати внимание: образ `quay.io/minio/minio` (MinIO ушёл с Docker Hub).

Проверка: открой `http://localhost:9001` → логин `minioadmin`, пароль `minioadmin`. Должна открыться консоль MinIO.

Создай bucket `warehouse` (сюда Trino будет класть таблицы). В консоли: слева **Buckets** → **Create Bucket** → имя `warehouse` → **Create Bucket**.

Альтернатива через командную строку (то же самое, что кнопка в UI):

```
docker run --rm --entrypoint /bin/sh --add-host host.docker.internal:host-gateway \  
quay.io/minio/mc:latest -c "mc alias set m http://host.docker.internal:9000 minioadmin minioadmin && mc mb m/warehouse"
```

4. Этап 2 — Lakekeeper (Iceberg REST Catalog)

Lakekeeper хранит свои метаданные в Postgres, поэтому поднимаем два контейнера: Postgres и сам Lakekeeper.

4.1. Postgres:

```
docker run -d --name lakehouse-pg \
  -e POSTGRES_PASSWORD=postgres \
  -p 5432:5432 \
  -v pg-data:/var/lib/postgresql/data \
  postgres:17
```

Подожди ~5 секунд, пока Postgres поднимется (проверка: `docker exec lakehouse-pg pg_isready` должно вывести `accepting connections`).

4.2. Миграции БД Lakekeeper (одноразовая команда, создаёт таблицы каталога):

```
docker run --rm \
  -e 'LAKEKEEPER__PG_ENCRYPTION_KEY=This-is-NOT-Secure!' \
  -e
  'LAKEKEEPER__PG_DATABASE_URL_READ=postgresql://postgres:postgres@host.docker.internal:5432/postgres' \
  -e
  'LAKEKEEPER__PG_DATABASE_URL_WRITE=postgresql://postgres:postgres@host.docker.internal:5432/postgres' \
  quay.io/lakekeeper/catalog:latest-main migrate
```

4.3. Сам Lakekeeper (REST-каталог на порту 8181):

```
docker run -d --name lakekeeper \
  -e 'LAKEKEEPER__PG_ENCRYPTION_KEY=This-is-NOT-Secure!' \
  -e
  'LAKEKEEPER__PG_DATABASE_URL_READ=postgresql://postgres:postgres@host.docker.internal:5432/postgres' \
  -e
  'LAKEKEEPER__PG_DATABASE_URL_WRITE=postgresql://postgres:postgres@host.docker.internal:5432/postgres' \
  -p 8181:8181 \
  quay.io/lakekeeper/catalog:latest-main serve
```

Проверка: `curl http://localhost:8181/health` должен вернуть `200` (эндпоинт здоровья каталога).

4.4. Принять Terms of Use и зарегистрировать warehouse demo (один раз, через API).

Сначала создай в рабочей папке файл `create-warehouse.json` с таким содержимым (он описывает, в какой бакет MinIO и под какими кредами Lakekeeper будет писать данные):

```
{
  "warehouse-name": "demo",
  "project-id": "000000000-0000-0000-0000-000000000000",
  "storage-profile": {
    "type": "s3",
    "bucket": "warehouse",
    "key-prefix": "",
    "endpoint": "http://host.docker.internal:9000",
    "region": "us-east-1",
    "path-style-access": true,
    "flavor": "s3-compat",
    "sts-enabled": false
  },
  "storage-credential": {
    "type": "s3",
    "credential-type": "access-key",
    "access-key-id": "minioadmin",
    "secret-access-key": "minioadmin"
  }
}
```

Теперь прими Terms of Use (одноразовый шаг) и зарегистрируй warehouse:

```
curl -X POST http://localhost:8181/management/v1/bootstrap \
-H "Content-Type: application/json" \
-d '{"accept-terms-of-use": true}'

curl -X POST http://localhost:8181/management/v1/warehouse \
-H "Content-Type: application/json" \
-d @create-warehouse.json
```

- Первый запрос должен вернуть 204. Если вместо этого Catalog is not open for bootstrap / CatalogAlreadyBootstrapped — это норм: Lakekeeper уже инициализирован (например, ты принял ToU через UI), просто пропусти этот шаг.
- Второй запрос должен вернуть 201 Created со "status":"active".
- Если curl пишет Failed to open create-warehouse.json — ты запускаешь его не из рабочей папки. Сначала `cd ~/lakehouse`.

5. Этап 3 — Trino (SQL-движок)

Trino узнаёт о каталогах из файлов в папке `/etc/trino/catalog/`. Мы смонтируем туда свою локальную папку `catalog/` — так ты сможешь сам класть в неё `.properties`-файлы и менять

каталоги (добавил файл → перезапустил Trino → каталог появился).

5.1. Создай папку `catalog/`. В рабочей папке (`~/lakehouse`) сделай подпапку `catalog` — в неё будем класть файлы каталогов:

```
mkdir -p catalog
```

5.2. Создай файл каталога. В `catalog/` положи файл `lakekeeper.properties` с таким содержимым (имя файла `lakekeeper` — это и есть имя каталога в Trino):

```
connector.name=iceberg
iceberg.catalog.type=rest
iceberg.rest-catalog.uri=http://host.docker.internal:8181/catalog
iceberg.rest-catalog.warehouse=demo
iceberg.rest-catalog.nested-namespaces-enabled=true
iceberg.unique-table-location=true
fs.native-s3.enabled=true
s3.endpoint=http://host.docker.internal:9000
s3.path-style-access=true
s3.region=us-east-1
s3.aws-access-key=minioadmin
s3.aws-secret-key=minioadmin
```

⚠ Ключевая строка — `fs.native-s3.enabled=true` (именно `fs.native-s3`, не `iceberg.s3` и не `fs.s3`). Без неё Trino падает на старте с ошибкой «неизвестные property».

5.3. Запусти Trino, смонтировав папку целиком (из рабочей папки, где лежит `catalog/`):

```
docker run -d --name trino \
  -p 8080:8080 \
  -v "$(pwd)/catalog:/etc/trino/catalog" \
  trinodb/trino:476
```

Проверка: `curl http://localhost:8080/v1/info` вернёт JSON; через ~20–30 секунд `docker ps` покажет Trino как `healthy`.

💡 **Поменял файл в `catalog/` или добавил новый `.properties`?** Пересоздавать контейнер не нужно — хватит перезапуска: `docker restart trino` (или кнопка **Restart** в Docker Desktop). Trino читает каталоги при старте. `docker rm -f + docker run` нужен только когда меняешь сам mount `-v` (другую папку).

6. Начать пользоваться: DBeaver

1. Скачай DBeaver: <https://dbeaver.io/download/> (Community Edition достаточно).

2. Database → New Database Connection → Trino.

3. Параметры: - Host: localhost , Port: 8080 - Database/Schema (catalog): lakekeeper - Username: trino , Password: пусто - JDBC URL получится: jdbc:trino://localhost:8080/lakekeeper

4. Test Connection → Finish.

Проверь связку, выполнив в SQL-редакторе DBeaver:

```
SHOW CATALOGS;                -- увидишь `lakekeeper`
CREATE SCHEMA lakekeeper.lakehouse;    -- создаём схему
CREATE TABLE lakekeeper.lakehouse.people (
    id    bigint,
    name  varchar
);
INSERT INTO lakekeeper.lakehouse.people VALUES (1, 'alice'), (2, 'bob');
SELECT * FROM lakekeeper.lakehouse.people;    -- должны вернуться 2 строки
```

Если всё сработало — загляни в консоль MinIO (<http://localhost:9001>) → bucket warehouse . Там появятся папки вида people-`<uuid>`/data/*.parquet и ../metadata/*.json — это и есть Lakehouse: данные лежат parquet-файлами в объектном хранилище, а Iceberg-метаданные рядом.

7. Остановить и удалить всё

Полная очистка (контейнеры + данные + образы) — выполняй команды по порядку:

```
# 1. Контейнеры (флаг -f останавливает и удаляет сразу)
docker rm -f trino lakekeeper lakehouse-pg lakehouse-minio

# 2. Данные (тома MinIO и Postgres)
docker volume rm minio-data pg-data

# 3. Образы (скачаются заново при следующем запуске)
docker rmi trinodb/trino:476 \
    quay.io/lakekeeper/catalog:latest-main \
    postgres:17 \
    quay.io/minio/mc:latest \
    quay.io/minio/minio:latest
```

Проверка, что всё удалилось:

```
docker ps -a    # не должно быть trino / lakekeeper / lakehouse-*
docker volume ls # не должно быть minio-data / pg-data
docker images    # не должно быть minio / lakekeeper / postgres:17 / trinodb
```

Чтобы **просто приостановить** (данные сохраняются, потом продолжишь с того же места) — `docker stop trino lakekeeper lakehouse-pg lakehouse-minio` без `rm`.

8. Если что-то пошло не так

- **Порт занят** (`port is already allocated`). Освободи порт или поменяй `-p 5432:5432` → `-p 5433:5432` (и тогда во всех `DATABASE_URL :5432` → `:5433`).
 - **Lakekeeper: Error creating read/write pool / pool timed out while waiting for an open connection** — Lakekeeper не достучался до Postgres. Почти всегда это значит, что пропущен шаг 4.1 (запуск Postgres). Подними Postgres, дождись `docker exec lakehouse-pg pg_isready`, затем повтори `migrate` и `serve`.
 - **minio/minio: not found / pull access denied** — MinIO убрали с Docker Hub, используй `quay.io/minio/minio`.
 - **Trino упал на старте** — смотри логи: `docker logs trino`. Если ругается на `iceberg.s3.*` или `fs.s3.*` — в конфиге неверный префикс, нужен `fs.native-s3.enabled=true` + `s3.*`.
 - **Trino не видит данные в MinIO** — проверь, что bucket называется именно `warehouse` и что `warehouse demo` зарегистрирован (повтори 4.4).
 - **host.docker.internal не резолвится** (обычно бывает на Linux, не на Docker Desktop) — добавь флаг `--add-host host.docker.internal:host-gateway` к каждому `docker run`.
-